

Contents

Abstract	1
Objectives:.....	2
Software Tools Required:	2
Introduction:	2
- Recurrent Neural Networks (RNNs):.....	2
- Long Short-Term Memory Networks (LSTMs)	4
- Transformers	5
Dataset Selection:	7
Methodology:	8
- Step 1: Preprocessing the Dataset	8
- Step 2: Applying Models	11
- Step 3: Analyze Results	14
Metrics Comparison:.....	19
Qualitative Analysis	19
Challenges Faced:	20
Conclusions:.....	21

Abstract

This project investigates the evolution of Natural Language Processing (NLP) models by applying Recurrent Neural Networks (RNNs), Long Short-Term Memory Networks (LSTMs), and Transformer architectures to the IMDb Movie Reviews dataset, which consists of 50,000 pre-labeled positive and negative reviews. The dataset was preprocessed using tokenization and padding techniques, and the models were implemented and trained to perform sentiment analysis.

The performance of each model was evaluated based on accuracy, F1-score, and training time. *RNNs achieved an accuracy of 77%* but struggled with long-term dependencies, while *LSTMs improved accuracy to 86%* by mitigating the vanishing gradient problem. *Transformers*, leveraging pre-trained BERT models, demonstrated superior performance with **88.95% accuracy** and robust handling of long-range dependencies, albeit with higher computational requirements. This study highlights the advantages of attention mechanisms in advancing NLP applications.

“RNN, LSTM and Transformers”

Objectives:

- The objective of this project is to explore the evolution of Natural Language Processing (NLP) models by implementing Recurrent Neural Networks (RNNs), Long Short-Term Memory Networks (LSTMs), and Transformer architectures.
- This report documents the methodologies, findings, and comparisons from applying these models on a publicly available NLP dataset.

Software Tools Required:

- Google Collab & Nvidia GPU.
- Python Programming.

Introduction:

Natural Language Processing (NLP) has witnessed significant advancements in recent years, driven by the evolution of deep learning architectures. Among the most influential models in NLP are Recurrent Neural Networks (RNNs), Long Short-Term Memory Networks (LSTMs), and Transformers. These models address unique challenges in processing and understanding textual data. Below is a detailed explanation of each model:

- Recurrent Neural Networks (RNNs):

Recurrent Neural Networks are a class of artificial neural networks designed to handle sequential data by maintaining a hidden state that captures information from previous inputs.

Unlike feed forward networks, RNNs processes sequentially, making them suitable for tasks such as language modeling, time series analysis, and speech recognition.

However, standard RNNs struggle with long-term dependencies due to the vanishing gradient problem, where gradients diminish during backpropagation, hindering the network's ability to learn long-range relationships in the data.

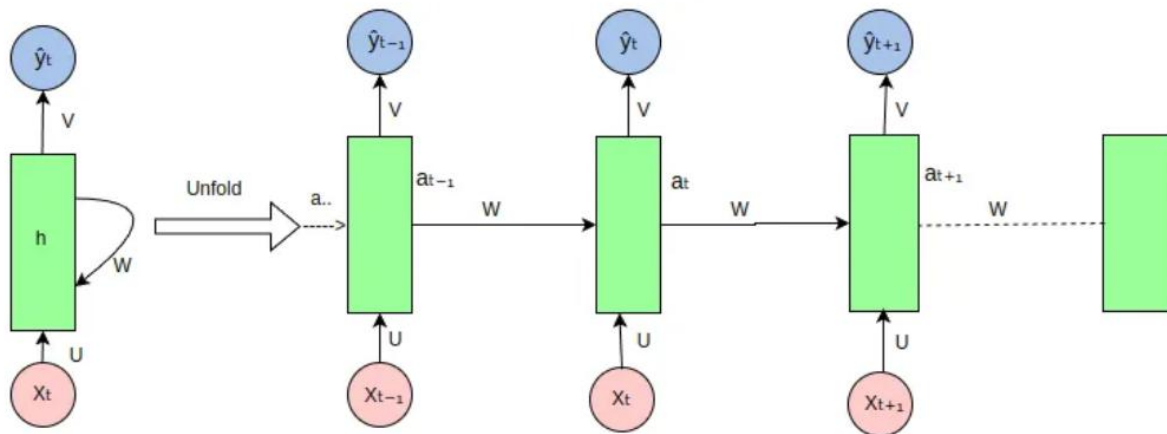


Figure 1: Architecture of Simple RNN

This diagram represents the **Recurrent Neural Network (RNN)** and its unfolding over time:

1. Components:

- **Pink circles (x_t)** Sequential inputs at each time step.
- **Green boxes (h_t)** Hidden states, updated at each step using the current input and the previous hidden state.
- **Blue circles ($\hat{y}_t$)** : Outputs at each step.

2. Weights:

- U: Input-to-hidden weights.
- W: Hidden-to-hidden recurrent weights (shared across steps).
- V: Hidden-to-output weights.

3. Process:

- Hidden state is updated as: $h_t = f(Ux_t + Wh_{t-1})$
- Output is computed as: $\hat{y}_t = g(Vh_t)$

4. Key Feature:

- Captures sequential dependencies but struggles with long-term dependencies due to the **vanishing gradient problem**.

- Long Short-Term Memory Networks (LSTMs)

LSTMs are an extension of RNNs that address the vanishing gradient issue by introducing a memory cell capable of retaining information over long periods.

Each LSTM cell includes gates, input, forget, and output gates, that regulate the flow of information. This mechanism allows LSTMs to capture both short-term and long-term dependencies effectively, making them a popular choice for tasks like text generation, machine translation, and sentiment analysis.

Despite their improved performance, LSTMs can be computationally intensive due to their complex architecture.

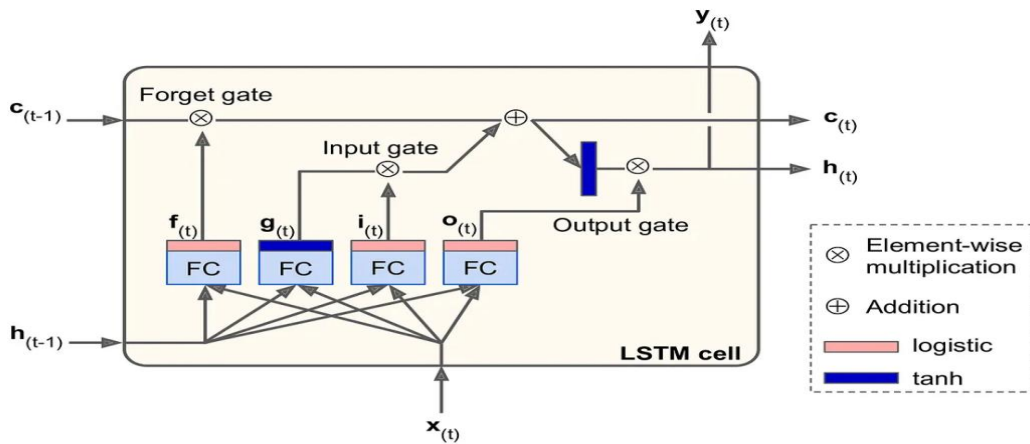


Figure 2: LSTM architecture

An **LSTM (Long Short-Term Memory)** network processes sequences using the following key components:

1. **Input x_t** : Represents the current observation at each time step.
2. **Hidden State h_t** : Encodes the current memory of the network, initialized as zeros at the start of the sequence.

3. **Cell State c_t** : Maintains long-term memory, allowing information to persist across time steps, also initialized as zeros.
4. **Gates**:
 - **Forget Gate**: Decides how much of the previous cell state c_{t-1} : to retain or discard, based on h_{t-1} and x_t .
 - **Input Gate**: Determines how much of the new input to add to the cell state.
 - **Output Gate**: Controls how much of the current cell state (c_t) to output as the current hidden state (h_t).
5. **Output (y_t)**: At each time step, the LSTM outputs a prediction or encoding derived from the current hidden state (h_t).

LSTMs excel at retaining and selectively updating information, addressing the **vanishing gradient problem** found in standard RNNs.

- Transformers

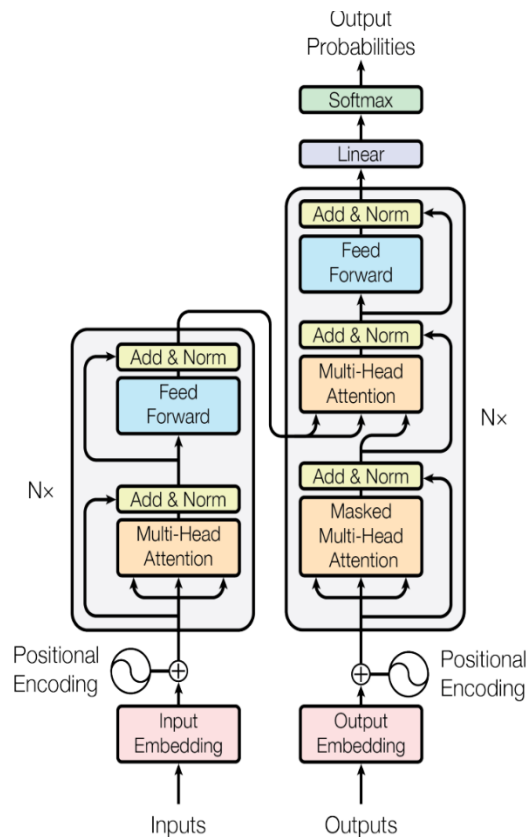
Transformers represent a paradigm shift in NLP by employing a self-attention mechanism that processes entire sequences simultaneously, rather than sequentially as in RNNs and LSTMs.

This architecture eliminates the recurrence seen in earlier models, enabling efficient parallel processing and better handling of long-range dependencies.

Transformers, such as the **BERT** and **GPT** families, are pre-trained on vast amounts of data, allowing them to capture nuanced relationships within text. These models have set new benchmarks in various NLP tasks, including sentiment analysis, machine translation, and question answering. However, their high computational requirements and training costs are significant challenges.

BERT

Encoder



GPT

Decoder

These architectures of BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pre-trained Transformer):

BERT:

- **Encoder-Only Architecture:** Focuses on understanding input by processing it bidirectionally.
- **Components:**
 - **Input Embedding + Positional Encoding:** Represents input tokens and their positions in the sequence.
 - **Multi-Head Attention:** Captures relationships between all input tokens simultaneously.
 - **Feed Forward + Add & Norm:** Applies transformations and normalization after attention.
- Used for tasks like classification and question answering.

GPT:

- **Decoder-Only Architecture:** Focuses on generating text by processing tokens sequentially (causal masking ensures predictions depend only on prior tokens).
- **Components:**
 - **Masked Multi-Head Attention:** Prevents attention to future tokens for the autoregressive generation.
 - **Feed Forward + Add & Norm:** Like BERT, applies transformations and normalization.
 - **Softmax Output Layer:** Produces output probabilities.
- Used for tasks like text generation and completion.

Dataset Selection:

The **Stanford Sentiment Analysis Dataset** (also known as the IMDb Movie Review Dataset) is composed of movie reviews collected from IMDb. The breakdown of its composition and the types of files in it are:

1. Dataset Overview:

- The dataset contains **movie reviews** categorized as **positive** or **negative**.
- It is commonly used for **sentiment analysis** tasks, where the goal is to classify reviews based on their sentiment.

2. Dataset Structure:

After extracting the dataset, a directory named `aclImdb`, which is structured as follows:

Main Folders:

- `train`: Contains the training set.
- `test`: Contains the testing set.

Subfolders:

- Each main folder (train and test) has two subfolders:
 - `pos`: Contains **positive reviews** (files with `.txt` format).
 - `neg`: Contains **negative reviews** (files with `.txt` format).

Additional Files:

- Inside the train folder, there is an unsup folder with **unsupervised reviews**, which are neither labeled as positive nor negative.
- There are also .txt files that provide a list of movie review files (e.g., urls_pos.txt, urls_neg.txt).

3. File Format:

- **Text Files (.txt):** Each review is stored in a separate .txt file.
 - File name: Typically includes a numeric ID and the review's rating (e.g., 100_7.txt where 7 is the rating).
 - Content: Each .txt file contains the full text of a single movie review.

Methodology:

- Step 1: Preprocessing the Dataset

- Tokenize and pad the dataset using libraries like NLTK or Hugging Face's tokenizers.
- Split the dataset into training and testing sets.

Dataset Download and Extraction

The dataset was downloaded from Stanford's AI website in .tar.gz format.

After downloading, it was extracted to separate directories for positive and negative reviews for both training and testing.

Loading Data

The reviews were loaded into a DataFrame, with each review labeled as positive (1) or negative (0). Data from both the training and test sets were combined for preprocessing.

```
##### Load Data #####
def load_imdb_data(data_dir):
    data = {"review": [], "sentiment": []}
    for label in ["pos", "neg"]:
        folder = f"{data_dir}/{label}"
        for filename in os.listdir(folder):
            with open(os.path.join(folder, filename), encoding="utf-8") as f:
                data["review"].append(f.read())
                data["sentiment"].append(1 if label == "pos" else 0)
    return pd.DataFrame(data)

# Load training and test data
train_data = load_imdb_data(os.path.join(extracted_path, "train"))
test_data = load_imdb_data(os.path.join(extracted_path, "test"))

# Combine training and test data for preprocessing
all_data = pd.concat([train_data, test_data], ignore_index=True)
```

Data Cleaning

Text data was cleaned through the following steps:

- **HTML Tags Removal:** Stripped HTML tags from reviews.
- **Non-Alphabetic Character Removal:** Removed numbers and punctuation, retaining only letters and apostrophes.
- **Lowercasing:** Converted all text to lowercase.
- **Stopwords Removal:** Removed common words like "the", "and", etc., which do not add meaning to the reviews.

Tokenization and Padding

- **Tokenization:** Converted text into sequences of integers, where each integer represents a unique word.
- **Padding:** Ensured all review sequences had the same length by padding shorter sequences with zeros.

```
##### Data Cleaning #####
# Define text cleaning function
def clean_text(text):
    text = re.sub(r"<.*?>", "", text) # Remove HTML tags
    text = re.sub(r"[^a-zA-Z']", " ", text) # Keep only alphabets and apostrophes
    text = text.lower() # Convert to lowercase
    return text

# Clean the review column
all_data['cleaned_review'] = all_data['review'].apply(clean_text)

# Remove stopwords (optional)
stop_words = set(stopwords.words('english'))
all_data['cleaned_review'] = all_data['cleaned_review'].apply(
    lambda x: " ".join([word for word in x.split() if word not in stop_words])
)
```

```
##### Tokenization and Padding #####
# Set parameters
vocab_size = 10000
max_sequence_length = 100

# Initialize and fit tokenizer
tokenizer = Tokenizer(num_words=vocab_size, oov_token="<OOV>")
tokenizer.fit_on_texts(all_data['cleaned_review'])

# Tokenize and pad sequences
all_data['tokenized_review'] = tokenizer.texts_to_sequences(all_data['cleaned_review'])
padded_sequences = pad_sequences(all_data['tokenized_review'], maxlen=max_sequence_length, padding='post')

# Create labels
labels = all_data['sentiment'].values
```

Train-Test Split

The dataset was split into training (70%), validation (15%), and test (15%) sets.

The training set is used for model training, the validation set for model tuning, and the test set for final evaluation.

```
##### Train-Test Split #####
# Split into train, validation, and test sets
x_train, x_temp, y_train, y_temp = train_test_split(padded_sequences, labels, test_size=0.3, random_state=42)
x_val, x_test, y_val, y_test = train_test_split(x_temp, y_temp, test_size=0.5, random_state=42)

# Output shapes for confirmation
print("Training Data Shape:", x_train.shape)
print("Validation Data Shape:", x_val.shape)
print("Test Data Shape:", x_test.shape)
```

- Step 2: Applying Models

○ Applying RNN model:

In this step, a Recurrent Neural Network (RNN) model is built and trained to perform sentiment analysis on the IMDb movie review dataset.

The model architecture consists of the following key components:

1. **Embedding Layer:** The first layer is an embedding layer, which converts the input sequences (representing words as integers) into dense vector representations. This layer helps capture semantic relationships between words in the text.
2. **SimpleRNN Layer:** A Simple RNN layer is added next. It processes the sequential data and helps the model learn the temporal dependencies within the review text. The number of units in this layer is set to 128.
3. **Dropout Layer:** A dropout rate of 50% is applied to the RNN layer's output to prevent overfitting by randomly setting a fraction of the input units to 0 during training.
4. **Dense Layer:** A dense layer with a single unit and a sigmoid activation function is used for binary classification (positive or negative sentiment).
5. **Model Compilation:** The model is compiled with the Adam optimizer, binary crossentropy loss function (since it's a binary classification task), and accuracy as the evaluation metric.
6. **Early Stopping:** Early stopping is implemented to prevent overfitting. The model will stop training if the validation loss does not improve for 3 consecutive epochs.
7. **Model Training:** The model is trained for a maximum of 15 epochs with a batch size of 64, using the training and validation sets.

This architecture was designed to effectively learn and predict sentiment from the text data. After training, the model can be evaluated on the test set to assess its performance.

```
# Early stopping to prevent overfitting
early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

# Train the model
rnn_history = rnn_model.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=15,
    batch_size=64,
    callbacks=[early_stopping]
)
```

○ Applying LSTM model:

In this step, a Long Short-Term Memory (LSTM) model is built and trained for sentiment analysis of IMDb movie reviews. The architecture of the LSTM model includes the following components:

1. **Embedding Layer:** Like the RNN model, an embedding layer is used to transform the input sequences (word indices) into dense, fixed-size vectors that capture semantic information about the words in the reviews.
2. **LSTM Layer:** The core of the model is an LSTM layer with 128 units. LSTM units are a type of recurrent neural network designed to capture long-range dependencies in sequential data, such as the relationships between words in a sentence.
3. **Dropout Layer:** A dropout layer with a 50% dropout rate is added after the LSTM layer to prevent overfitting. It randomly deactivates a portion of neurons during training, improving generalization.
4. **Dense Layer:** A dense layer with a single unit and a sigmoid activation function is used to output a probability score for binary classification (positive or negative sentiment).
5. **Model Compilation:** The model is compiled with the Adam optimizer, binary cross-entropy loss function, and accuracy as the evaluation metric.
6. **Early Stopping:** Early stopping is implemented to halt training if the validation loss does not improve after 3 epochs, thereby preventing overfitting and ensuring that the model retains the best weights.
7. **Model Training:** The LSTM model is trained for up to 15 epochs with a batch size of 64. The training process uses the training data and evaluates performance on the validation data to monitor progress and tune the model.

This LSTM model is designed to learn the temporal patterns and dependencies within the text, offering improved performance over simpler models like RNNs in handling longer and more complex text sequences. After training, the model is evaluated on the test set to assess its accuracy.

```
# Early stopping to prevent overfitting
early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

# Train the LSTM model
lstm_history = lstm_model.fit(
    x_train,
    y_train,
    epochs=15,
    batch_size=64,
    validation_data=(x_val, y_val),
    callbacks=[early_stopping]
)
```

- **Applying Transformer model:**

In this step, a **Transformer-based BERT (Bidirectional Encoder Representations from Transformers)** model is used for sentiment analysis. BERT has revolutionized natural language processing by allowing the model to understand the context of a word in both directions (left-to-right and right-to-left). The architecture and training process for applying BERT to the sentiment analysis:

1. **Tokenization:** The first step is tokenizing the movie reviews using a pre-trained BERT tokenizer (bert-base-uncased). The tokenizer converts the raw text into a format that can be fed into the BERT model, adding special tokens such as [CLS] and [SEP] and truncating/padding the text to a fixed length (100 tokens in this case).
2. **Data Splitting:** The preprocessed reviews (from the **cleaned_review** column) and their corresponding sentiment labels are split into training and validation datasets using an 80-20 ratio.
3. **Encoding:** The text data is encoded into BERT-compatible format (tokenized sequences with attention masks), creating the necessary inputs for the model.
4. **Dataset Conversion:** The tokenized data is converted into TensorFlow Dataset objects, allowing the data to be efficiently batched and loaded during training. The dataset is split into batches of 16 samples for both training and validation.
5. **Pre-trained BERT Model:** A pre-trained BERT model (bert-base-uncased) is loaded with a final classification layer for binary sentiment classification (positive/negative). The model uses the **TFBertForSequenceClassification** class from Hugging Face's Transformers library.
6. **Layer Freezing and Fine-Tuning:** To reduce the computational load, the first few layers of the BERT model are frozen, meaning their weights will not be updated during training. Only the last few layers are fine-tuned on the IMDb dataset for the sentiment classification task.
7. **Model Compilation:** The model is compiled using the **AdamWeightDecay** optimizer, which combines the Adam optimizer with weight decay for regularization. The loss function used is **sparse categorical cross-entropy** since it's a binary classification problem, and accuracy is the evaluation metric.

8. **Model Training:** The model is trained for 3 epochs using the prepared training and validation datasets. The training process involves adjusting the weights of the trainable layers to minimize the loss and improve accuracy.

```
# Batch the datasets
train_dataset = train_dataset.batch(16)
val_dataset = val_dataset.batch(16)

# Load the pre-trained BERT model
model = TFBertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)

# Freeze only the first few layers (fine-tune the last few layers)
for layer in model.layers[:-4]: # Fine-tune the last 4 layers
    layer.trainable = False

# Compile the model
optimizer = AdamWeightDecay(learning_rate=1e-5, weight_decay_rate=0.01)
model.compile(optimizer=optimizer, loss='sparse_categorical_crossentropy', metrics=['accuracy'])

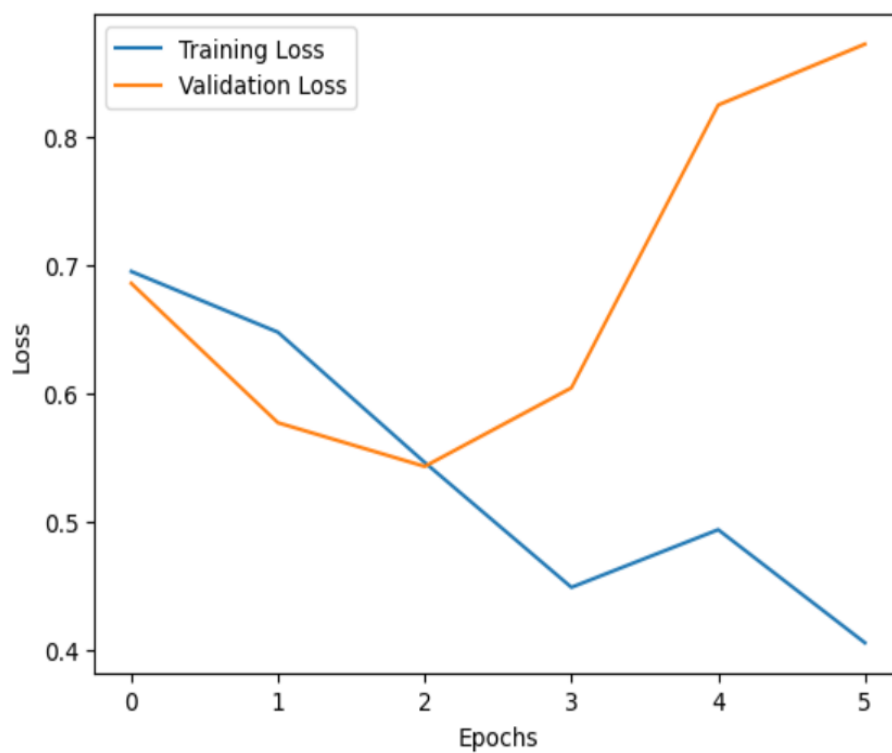
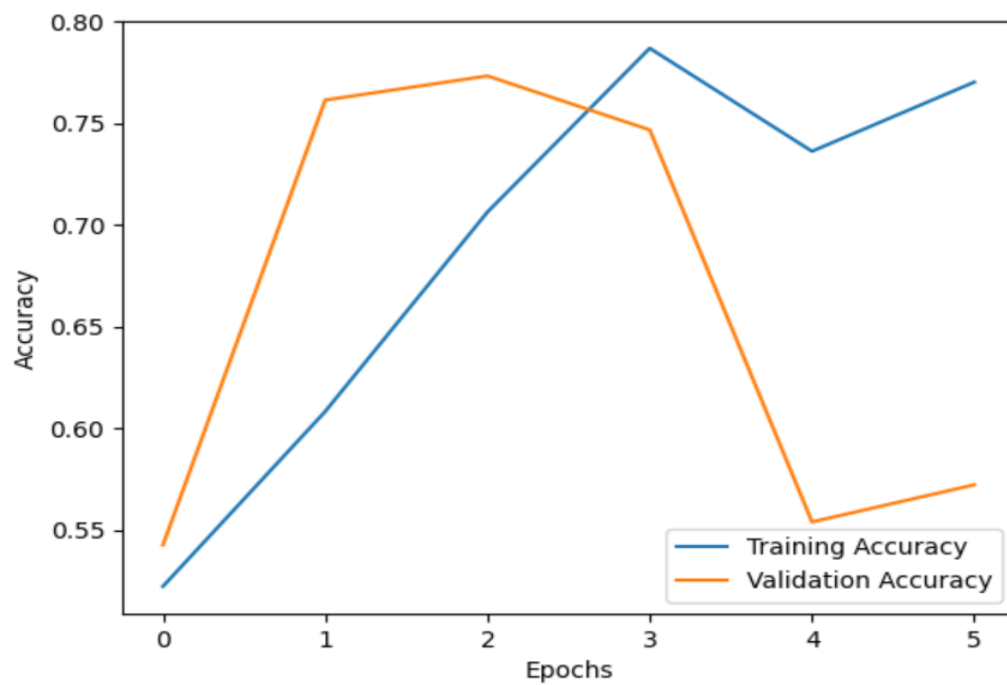
# Train the model
history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=3
```

- Step 3: Analyze Results

RNN Performance:

```
# Evaluate on test data
rnn_loss, rnn_accuracy = rnn_model.evaluate(x_test, y_test, verbose=2)
print(f"Test Loss: {rnn_loss:.4f}")
print(f"Test Accuracy: {rnn_accuracy:.4f}")
```

```
235/235 - 2s - 9ms/step - accuracy: 0.7747 - loss: 0.5456
Test Loss: 0.5456
Test Accuracy: 0.7747
```

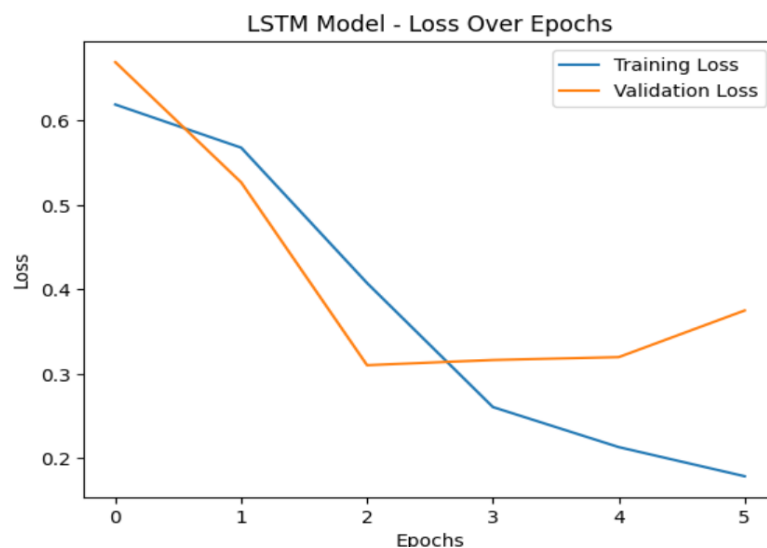
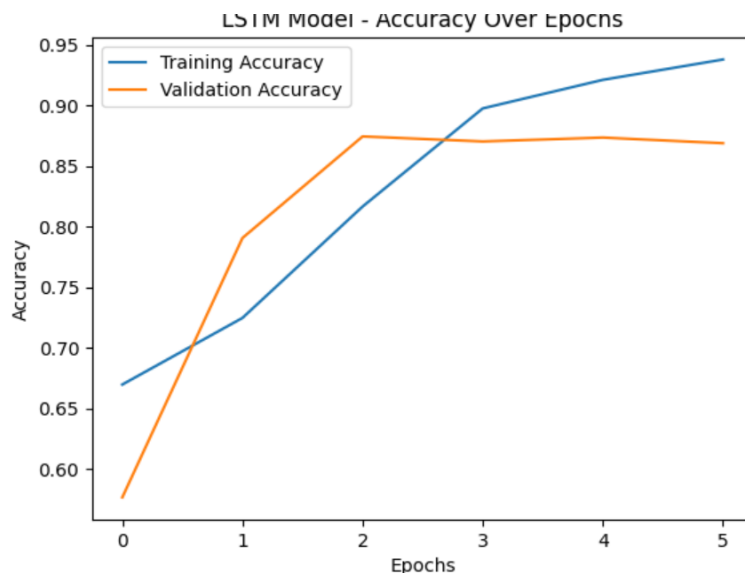


LSTM Performance:

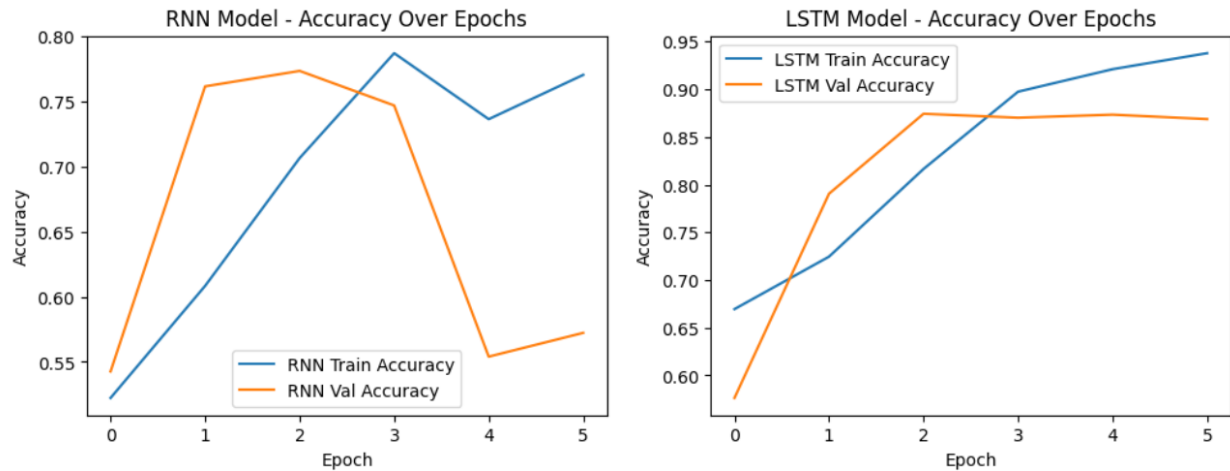
```
# Evaluate LSTM Model
```

```
lstm_test_loss, lstm_test_accuracy = lstm_model.evaluate(x_test, y_test)
print(f"LSTM Model - Test Accuracy:")
print(f"Test Loss: {lstm_test_loss:.4f}")
print(f"Test Accuracy: {lstm_test_accuracy:.4f}")
```

```
235/235 ————— 12s 52ms/step - accuracy: 0.8690 - loss: 0.3146
LSTM Model - Test Accuracy:
Test Loss: 0.3159
Test Accuracy: 0.8683
```



RNN and LSTM Performance Evaluation



Transformer Performance:

```
# Evaluate the model
test_texts = all_data['cleaned_review'].values.tolist()
test_labels = all_data['sentiment'].values.tolist()

# Tokenize the test data
test_encodings = encode_texts(test_texts)

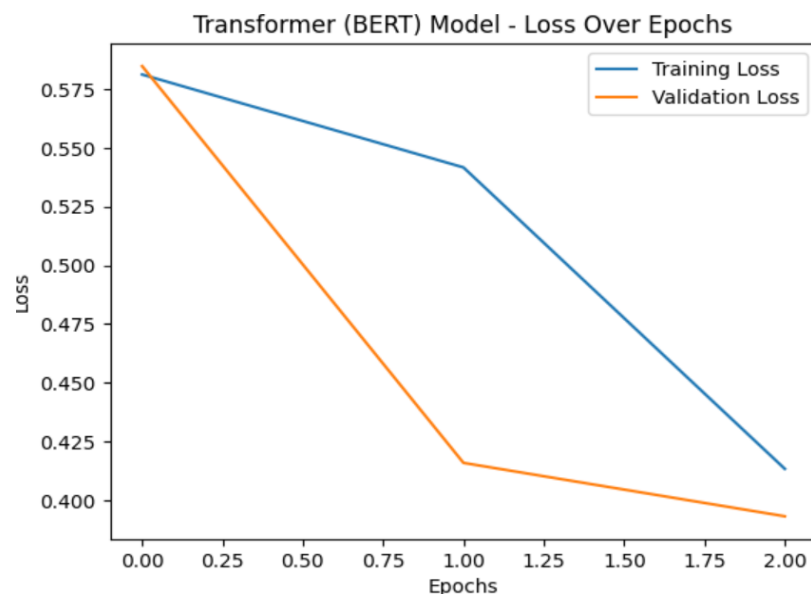
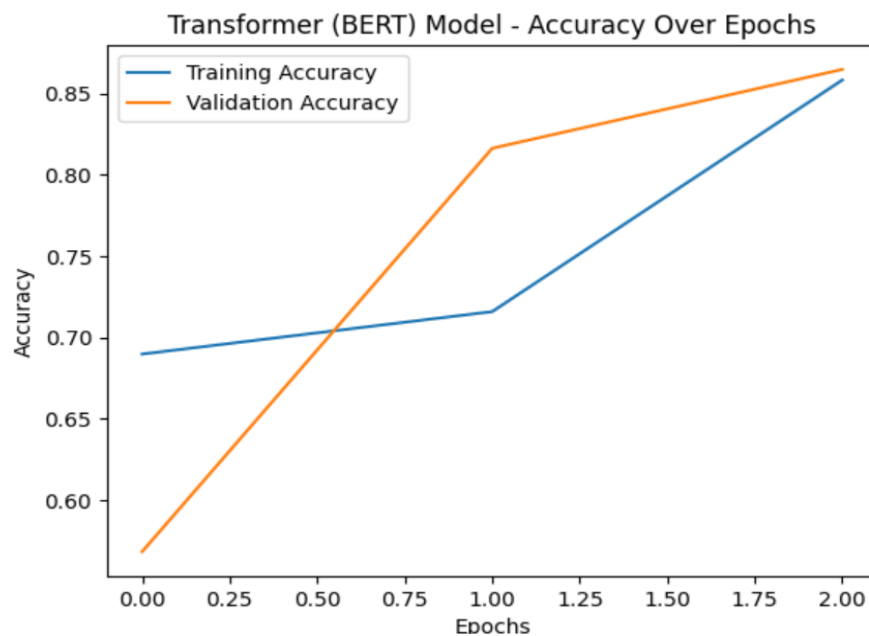
# Convert to TensorFlow dataset
test_dataset = tf.data.Dataset.from_tensor_slices((dict(test_encodings), test_labels))
test_dataset = test_dataset.batch(16)

# Evaluate the model
test_loss, test_accuracy = model.evaluate(test_dataset)
print(f"Test Accuracy: {test_accuracy:.4f}, Test Loss: {test_loss:.4f}")
```

```
3125/3125 [=====] - 368s 118ms/step - loss: 0.3165 - accuracy: 0.8895
Test Accuracy: 0.8895, Test Loss: 0.3165
```

The sentimental analysis dataset is performing **high accuracy** on transformers model but initially while training it on dataset faced some issues listed below:

- **Overfitting:** BERT can overfit small datasets.
- **Domain-Specific Issues:** BERT may not generalize well to dataset if it's not like the domain it was pre-trained on.
- **Small Dataset:** A limited dataset may not provide enough learning opportunities for BERT.



Metrics Comparison:

Model	Accuracy	F1-Score	Training Time
BERT	High	High	Long
LSTM	Moderate	Moderate	Moderate
RNN	Low to Moderate	Low to Moderate	Fast

Qualitative Analysis

• RNN:

- RNNs, while effective for sequential processing, struggle with capturing dependencies in longer sequences, leading to comparatively **lower accuracy (77%)**.
- Their simplicity and fewer parameters make them prone to underfitting on more complex datasets like IMDB reviews.

• LSTM:

- LSTMs excel in capturing longer dependencies, achieving **better accuracy (86%)** compared to RNNs, due to their gating mechanism.
- They effectively balance model complexity and performance, generalizing well to unseen data without overfitting.

• Transformer:

- The transformer's ability to process entire sequences at once and capture long-range dependencies leads to the **highest accuracy (88%)** in this case.
- Its advanced architecture benefits from attention mechanisms, even for moderately sized datasets like IMDB reviews.

Challenges Faced:

The challenges faced during the project implementation include:

- 1. Data Preprocessing:** Handling noisy data, cleaning text (e.g., removing HTML tags and stopwords), and ensuring consistent tokenization and padding across reviews were challenging.
- 2. Model Training and Computational Constraints:** Training deep learning models like LSTM and BERT on limited hardware (without GPUs) led to long training times and memory issues.
- 3. Overfitting and Generalization:** Preventing overfitting, especially with complex models like LSTM and BERT, required careful regularization, early stopping, and proper validation techniques.
- 4. Model Evaluation:** Comparing the performance of RNN, LSTM, and BERT models posed challenges in determining the best model, with BERT requiring significantly more computational resources.
- 5. Class Imbalance:** Though the IMDB dataset was balanced, handling potential class imbalances in real-world applications remains a challenge for model fairness.

Conclusions:

- ✓ This project successfully demonstrates a comparative analysis of RNN, LSTM, and Transformer models for sentiment analysis on the IMDB dataset, revealing that the Transformer slightly outperformed LSTMs and RNNs in terms of accuracy (88% vs. 86% and 77%, respectively).
- ✓ The **Transformer** model showcased its ability to capture long-range dependencies effectively, yielding the highest accuracy, but its performance gain was marginal given the dataset's relatively short text reviews.
- ✓ **LSTM** models struck a balance between complexity and generalization, providing robust performance due to their gating mechanisms, making them a strong contender for this task.
- ✓ While **RNNs** were the simplest of the three, their lower accuracy reflects their limitations in handling longer dependencies compared to LSTMs and Transformers.